

Today's Agenda:

Starting 7:05

1. Sum of all subarray sum
2. Maximum Sum Subarray of length K .

Q:- 1. Given an array of size N .
Calculate & return the total subarray sum of all the subarrays.

ex:- $\{ \underset{0}{3}, \underset{1}{2}, \underset{2}{5} \}$

Subarray	Sum
3	3
3, 2	5
3, 2, 5	10
2	2
2, 5	7
5	5
	<u>32</u>

Idea 1: Consider every subarray by fixing the S.I & E.I. Run a loop on that subarray to cal. the sum. Accumulate the sum in a global variable. Return the ans.

```

int allSubarraySum ( int[] arr, int N ) {
    int totalSum = 0;
    for ( s = 0 ; s < N ; s++ ) {
        for ( e = s ; e < N ; e++ ) {
            int sum = 0;
            for ( int i = s ; i <= e ; i++ ) {
                sum += arr[i];
            }
            totalSum += sum;
        }
    }
    return totalSum;
}

```

T.C $\rightarrow O(N^3)$
S.C $\rightarrow O(1)$.

Idea 2:- Use Prefix Sum to calculate the sum of a subarray in constant time.

```
int allSubarraySum (int[] arr, int N) {
```

```
    int pSum[] = pSum(arr, N);  
    int totalSum = 0;
```

```
    for (s = 0; s < N; s++) {  
        for (e = s; e < N; e++) {
```

```
            int sum = 0;
```

```
            if (s == 0) {
```

```
                sum = pSum[e];
```

```
            } else {
```

```
                sum = pSum[e] - pSum[s-1];
```

```
            }
```

```
            totalSum += sum;
```

```
        }
```

```
    }
```

```
    return totalSum;
```

```
}
```

T.C $\rightarrow O(N^2)$.

S.C $\rightarrow O(N)$

$\uparrow$
pSum:

Idea 3:- Avoid extra space of psum
by applying C.F. technique.

s	e	Sum = 0;	total Sum = 0
0	0	+ arr[0];	[0,0] + arr[0];
	1	+ arr[1];	[0,1] + arr[0] + arr[1];
	2	+ arr[2];	[0,2] + arr[0] + arr[1] + arr[2];
	3	Sum = 0;	[1,1] + arr[1];
1	1	+ arr[1]	

Code.

```
int allSubarraySum (int arr[], int N) {
```

```
    int totalSum = 0;    int sum;
```

```
    for (s = 0; s < N; s++) {
```

```
        sum = 0;
```

```
        for (e = s; e < N; e++) {
```

```
            sum += arr[e];
```

```
            totalSum += sum;
```

```
    }
    return totalSum;
```

```
}
```

T.C $\rightarrow O(N^2)$.

S.C $\rightarrow O(1)$.

Dry Run.

$$\text{arr} = \begin{matrix} & S \\ & 0 & 1 & 2 \\ \{ & 3, & 2, & 5 \}. \end{matrix}$$

S	e	Sum = 0	total Sum = 0
0	0	+ 3	+ 3
	1	+ 2	+ 5
	2	+ 5	+ 10
	3	= 0	
1	1	+ 2	+ 2
	2	+ 5	+ 7.
	3	= 0	
2	2	+ 5	+ 5
	3		<u>32</u>

~~3~~

$$N^3 \xrightarrow[\text{S.C} \rightarrow O(N)]{\text{pSum}} N^2 \xrightarrow[\text{S.C} \rightarrow O(1)]{\text{C.F.}} N^2$$

Idea 4:

Contribution Technique.

<u>Subarray</u>	<u>Sum</u>	
3	3	3
3, 2	5	3 2
3, 2, 5	10	3 2 5
2	2	2
2, 5	7	2 5
5	5	5
	<u>32</u>	<u>3 × 3 2 × 4 5 × 3</u>

$$= 9 + 8 + 15$$
$$= \underline{\underline{32}}$$

The factor in ~~red~~ represents the no. of subarrays in which a particular element is occurring.

If we are able to compute this factor for every element i of the array, we are done.

Q:1.

$\{ \overset{0}{3}, \overset{1}{-2}, \overset{2}{4}, \overset{3}{-1}, \overset{4}{2}, \overset{5}{6} \}$

Ans = 10.

$$i = 1; \quad N = 6;$$

$$(i+1) * (N-i)$$

$$= (1+1) * (6-1) \\ = 2 * 5 = \underline{10}$$

Q:2.

$\{ \overset{0}{3}, \overset{1}{-2}, \overset{2}{4}, \overset{3}{-1}, \overset{4}{2}, \overset{5}{6} \}$

Ans = 12.

$$i = 2; \quad N = 6$$

$$(i+1) * (N-i)$$

$$= (2+1) * (6-2) \\ = 3 * 4 = \underline{12}$$

B₁

B₃

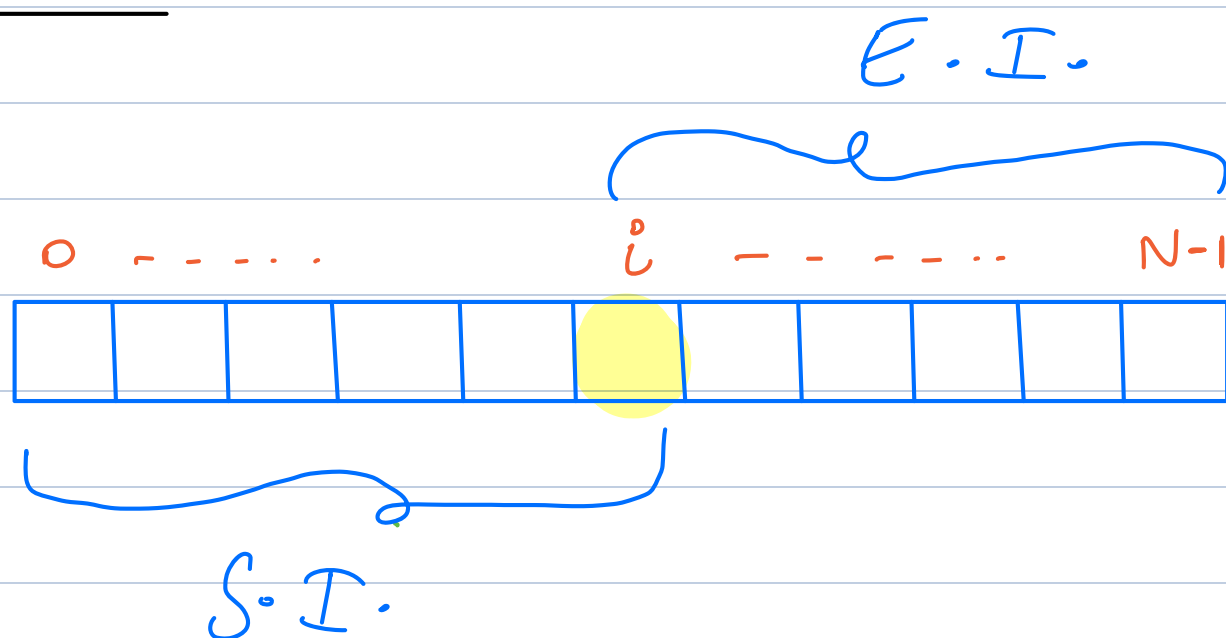
B₂

B₄

B₃

B₅

Generalisation



$$\text{Total S.I.} = [0, i] = (i+1)$$

$$\text{Total E.I.} = [i, N-1] = (N-i)$$

$\therefore$ Total Subarrays in which element at index i will be present

B_1 G_1
 B_2 G_2
 B_3 G_3
 B_4 G_4
 B_5

$$= (i+1) \times (N-i)$$

$$5 \times 4.$$

$\therefore$ Total Contribution of an element $A[i]$

=

$$A[i] \times (i+1) \times (N-i)$$

8:15

Optimised Code

```
int allSubarraySum (int arr[], int N) {
```

```
    int ans = 0;
```

```
    for (i = 0; i < N; i++) {
```

```
        ans += A[i] * (i+1) * (N-i);
```

```
    }
```

```
    return ans;
```

```
}
```

T.C $\rightarrow O(N)$

S.C $\rightarrow O(1)$

Break till 8:15.

Total Subarrays of Length K

arr = { ⁰3, ¹2, ²5, ³1, ⁴6, ⁵8, ⁶10 }

N = 7

<u>K</u>	<u>Subarrays</u>	<u>First S.I.</u>	<u>First E.I.</u>
1	7	0	0
2	6	0	1
3	5	0	2
4	4	0	3
5	3	0	4
6	2	0	5
7	1	0	6

Total Subarrays of
length $k = N - k + 1$

First Subarray

[0, k-1]

Q:- Given an array of size N & an integer K . Find maximum subarray sum of length K .

$$k = 3$$
[illegible] $[0, k-1]$

<u>id</u>	<u>e</u>	<u>sum</u>	
0	2	-1	→ <u>ans.</u>
1 ↘ +1	3 ↘ +1	7	
2 ↘ +1	4 ↘ +1	6	
3 ↘ +1	5 ↘ +1	6	
4	6		

Idea 1:- Consider every subarray of length k by fixing the starting & ending index. Iterate over that to compute the sum. Compare that sum with some global max sum & update the ans if you got a better sum.

Code

```
int maxSumSubarray (int arr[], int N,  
                    int k) {
```

```
    int ans = -∞ / INT_MIN;
```

```
    int s = 0, e = k-1;
```

```
    while (e < N) {
```

```
        int sum = 0;
```

```
        for (i = s; i ≤ e; i++) {  
            sum += arr[i];  
        }  
    }
```

```
        ans = max(ans, sum);
```

```
        s++; e++;
```

```
    }
```

```
    return ans;
```

3

$$(N-k+1) \times k.$$

1. $k=1$;

$$(N-1+1) \times 1 = N$$

2. $k=N$;

$$(N-N+1) \times N = N$$

$$3. \quad k = N/2; \quad (N - N/2 + 1) \times N/2 = \frac{N^2}{4}$$

$$N = 100$$

$$(100 - 50 + 1) \times 50$$

$$50 \times 50 = 2500$$

$$\therefore T.C = O(N^2)$$

Applying pSum:

$$T.C = N + (N - k + 1) \approx O(N)$$

$$S.C = O(N) \leftarrow pSum;$$

Optimisation

Sliding Window Approach

ex:- $\{-3, 4, -2, 5, 3, -2\}$

$\begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix}$

$$N = 6$$

$$K = 3$$

<u>i</u>	<u>e</u>	<u>Sum</u>
0	2	$A[0] + A[1] + A[2] = S$
1	3	$A[1] + A[2] + A[3] = S + A[3] - A[0]$
2	4	$A[2] + A[3] + A[4] = S + A[4] - A[1]$
3	5	$A[3] + A[4] + A[5] = S + A[5] - A[2]$

Moving from one subarray to the next there is only a change in 2 elements, one being added while other being removed.

If we know the summation of the base window, we can get the summation of next windows in constant time.

Code.

```
int maxSubarraySum (int arr, int N, int k) {
```

Cal. sum of base window.

```
    int sum = 0;
    for (int i = 0; i < k; i++) {
        sum += arr[i];
    }
    int ans = sum;
```

Move the window by 1 to optimally
Cal. the sum of remaining windows.

```
    int s = 1, e = k;
```

```
    while (e < N) {
```

```
        sum = sum + arr[e] - arr[s-1];
```

```
        ans = max(ans, sum);
```

```
        s++; e++;
```

3

N-k

3 return ans;

$$T.C \rightarrow \underline{\underline{O(N)}}$$

$$S.C \rightarrow \underline{\underline{O(1)}}$$

Any scenario where the subarray length is fixed, think about S.W. approach.